Strategy	Description
Fault containment or isolation	A fault should be contained within a part of the system and not escalate to a total crash.
Structure	Isolating a faulty component means a structure must exist to isolate the component from the rest of the system. The system needs a defined structure in which active parts can be isolated.
Redundancy	A backup component should be able to take over when a component fails.
Replacement	If a faulty component can be isolated, you can replace it in the structure. The other parts of the system should be able to communicate with the replaced component just as they did with the failed component.
Reboot	If a component gets into an incorrect state, you need the ability to get it back to a defined initial state. The incorrect state may be the reason for the fault, and it may not be possible to predict all the incorrect states the component can get into because of dependencies outside of your control.
Component lifecycle	A faulty component needs to be isolated. If it can't recover, it should be terminated and removed from the system or reinitialized with a correct starting state. A defined lifecycle must exist to start, restart, and terminate the component.
Suspend	When a component fails, all calls to the component should be suspended until the component is fixed or replaced so the new component can continue the work without dropping a beat. The call handled at the time of failure also should not disappear—it could be critical to recovery. Further, it may contain information critical to understanding why the component failed. You may want to retry the call when you're sure there was another reason for the fault.
Separation of concerns	It's great if the fault-recovery code can be separated from the normal processing code. Fault recovery is a cross-cutting concern in the normal flow. A clear separation between normal flow and recovery flow simplifies the work that needs to be done. Changing the way the application recovers from faults is simpler if you achieve this clean separation.